# Vulnerability Assessment Report
# (Read-Only Security Review)

**Target Website:** http://demo.testfire.net

**Prepared by:** Lethabo Mokoti

Cyber Security Internship – Future Interns (2026)

**Assessment Type:** Passive Vulnerability Assessment

**Date:** 25 April 2026

# Table of Contents

Executive Summary.....3

Scope & Methodology.....4

Tool used.....5

Findings and Risk Ratings.....6

Evidence Screenshots.....8

Recommended Next Steps.....12

Conclusion.....13

# Executive Summary

This vulnerability assessment was conducted to evaluate the security posture of a publicly accessible website using ethical, read-only testing techniques. The purpose of the assessment was to identify common security weaknesses, explain their potential business impact, and provide practical remediation recommendations.

The assessment focused on identifying configuration-related security issues, such as missing security headers, insecure cookie settings, and transport security weaknesses. These issues are commonly observed in public-facing web applications and do not indicate an active security breach. However, if left unaddressed, they may increase the likelihood of exploitation over time.

The findings were classified using a risk-based approach (Low / Medium) to help prioritize remediation efforts. While no critical vulnerabilities were identified, several medium-risk issues were observed that could expose users to client-side attacks or data interception if combined with other weaknesses.

Overall, the website demonstrates a basic but improvable security posture. By implementing the recommended security controls and best practices, the organization can significantly reduce risk, improve user trust, and strengthen its overall defensive posture.

# Scope & Methodology

**Assessment Scope**

The vulnerability assessment was conducted on the following target:

- **Website Tested:** http://demo.testfire.net
- **Assessment Type:** Read-only, passive security assessment

**In Scope**

- Public-facing pages only
- Passive analysis techniques
- Header and cookie inspection
- Configuration and exposure review

**Out of Scope**

- Login or authenticated areas
- Exploitation or payload injection
- Brute-force or denial-of-service attacks
- Any actions that could impact availability

**High-Level Methodology**

The assessment was conducted using passive and read-only techniques.
Public-facing pages were analyzed to identify common security misconfigurations without exploiting any vulnerabilities or performing intrusive testing.

The assessment followed a structured, non-intrusive approach:

1. Reviewed publicly accessible pages for visible exposure
2. Performed passive security analysis using automated and manual techniques
3. Inspected HTTP response headers and cookie configurations
4. Identified security misconfigurations and missing protective controls
5. Classified findings based on potential business and security impact
6. Documented results with clear explanations and remediation guidance

# Tools Used

This assessment was conducted using a minimal set of widely accepted security analysis tools. All tools were used in **passive mode only**, in line with ethical and read-only testing requirements.

**Tools Used**

- **OWASP ZAP (Passive Scan)**
  Used to identify passive security issues such as missing HTTP security headers, insecure cookie attributes, and general security misconfigurations without actively attacking the application.
- **Browser Developer Tools**
  Used to inspect HTTP response headers, cookies, and client-side security controls directly from the browser.
- **Nmap (Basic Scanning)**
  Used for limited visibility into exposed services and basic network-level information without intrusive probing or exploitation.

# Findings & Risk Ratings

This section presents the security weaknesses identified during the passive vulnerability assessment. All findings were categorized based on their **potential business and security impact** using a risk-based approach.

**Identified Vulnerabilities**

| Vulnerability | Risk Level |
|---|---|
| Missing Content Security Policy (CSP) | Medium |
| Missing X-Frame-Options | Low |
| Missing X-Content-Type-Options | Low |
| Missing HSTS / HTTPS Not Enforced | Medium |
| Insecure Cookie Flags | Medium |
| Server Version Disclosure | Low |
| Missing Referrer Policy | Low |

**Risk Classification Approach**

Risk levels were assigned based on the following criteria:

- **Low Risk**
  Issues that do not pose an immediate threat but may contribute to security weaknesses if combined with other vulnerabilities.
- **Medium Risk**
  Issues that could be exploited under certain conditions and may impact user security, data protection, or trust if left unresolved.

No **High-risk** or **critical** vulnerabilities were identified during this read-only assessment.

**Findings Overview**

The assessment identified several **configuration-related security weaknesses**, primarily involving missing or improper security controls at the HTTP and browser level.

While these issues are common in many web applications, addressing them would:

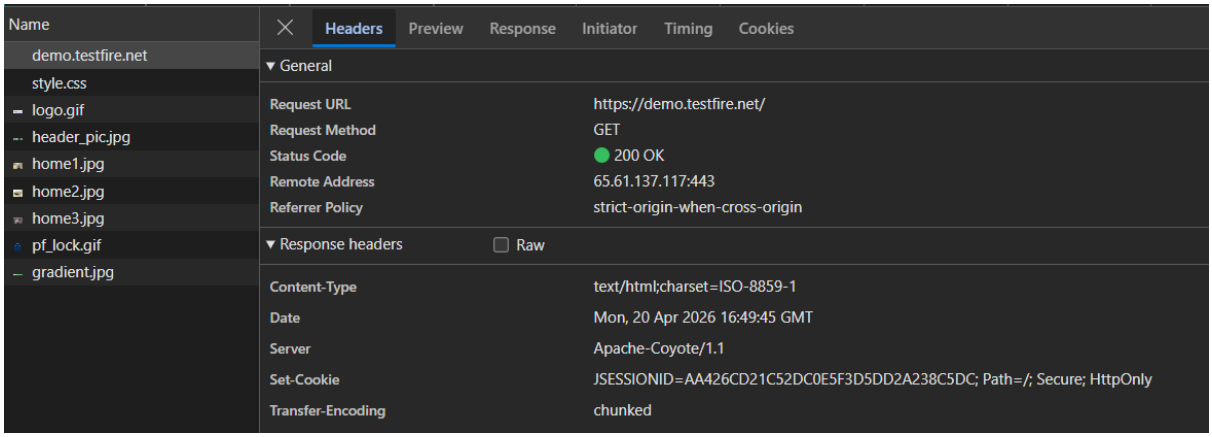
- Reduce the website's overall attack surface
- Improve protection against client-side attacks
- Align the application with modern web security best practices

The identified vulnerabilities can be remediated through **low-effort configuration changes** that provide **high security value**.

# Evidence Screenshots

**Tool:** Browser Developer Tools
**What to Capture:**

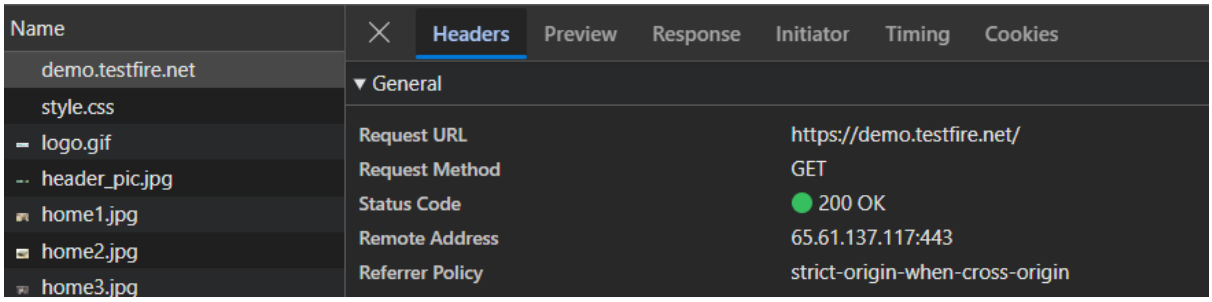
- Network tab
- Response headers
- Show missing:
  - Content-Security-Policy
  - X-Frame-Options
  - X-Content-Type-Options
  - Referrer-Policy




*headers_missing.png: HTTP response headers showing missing security headers, increasing exposure to common client-side web attacks.*

**Tool:** Browser DevTools
**What to Capture:**

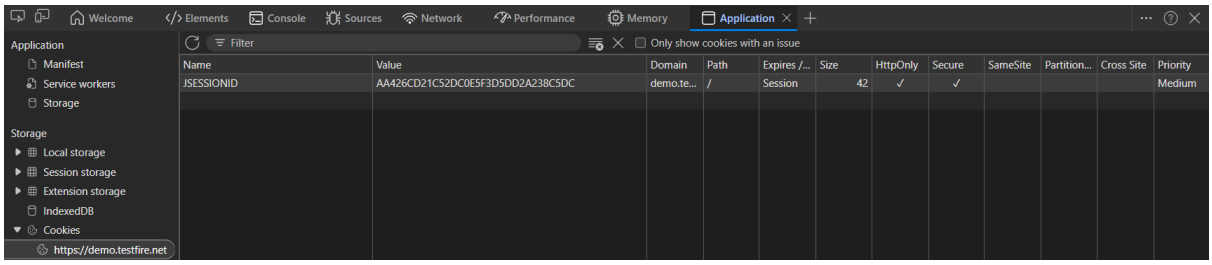
- Page loaded over HTTP or
- Response headers showing no Strict-Transport-Security




*https_hsts.png: Observation of HTTPS configuration showing absence of enforced HTTPS and missing HSTS protection.*

**Tool:** Browser DevTools (Application / Cookies tab)
**What to Capture:**

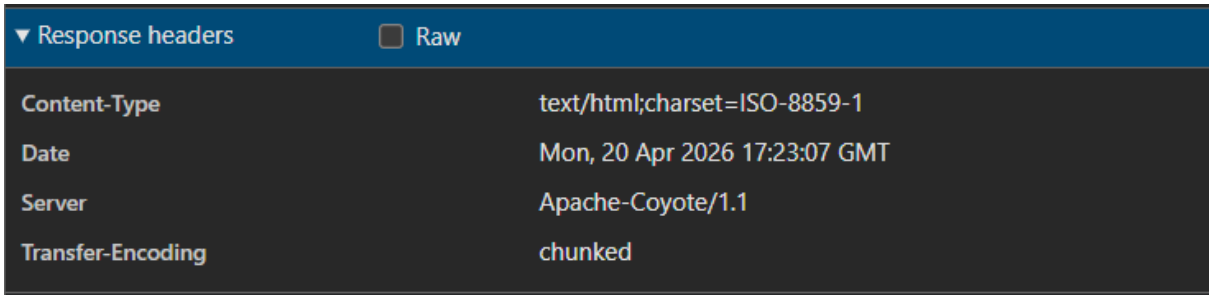
- Cookies without:
  - Secure
  - HttpOnly
  - SameSite




*cookies_flags.png: Cookies observed without recommended security attributes, which may increase the risk of session-related attacks.*

**Tool:** Browser DevTools
**What to Capture:**

- Response header revealing server or technology version



▼ Response headers  Raw

| | |
|---|---|
| Content-Type | text/html;charset=ISO-8859-1 |
| Date | Mon, 20 Apr 2026 17:23:07 GMT |
| Server | Apache-Coyote/1.1 |
| Transfer-Encoding | chunked |

*server_disclosure.png: HTTP response headers revealing server version information that could assist attackers in targeting known vulnerabilities.*

All evidence was collected using passive inspection techniques and does not represent exploitation or active testing.

# Recommended Next Steps

To improve the security posture of the website, it is recommended that the organization:

- Implement missing HTTP security headers such as CSP, X-Frame-Options, and Referrer-Policy
- Enforce HTTPS across all pages and enable HSTS
- Secure cookies by applying appropriate Secure, HttpOnly, and SameSite attributes
- Minimize information disclosure by suppressing server version details
- Conduct periodic security reviews to ensure configurations remain aligned with best practices

These actions are **low-cost**, **low-effort**, and provide **high security value**, making them suitable for immediate implementation.

# Conclusion & Next Steps

This vulnerability assessment was conducted using ethical, read-only security testing techniques on a publicly accessible demo website. The objective was to identify common security weaknesses, assess their potential impact, and provide practical remediation guidance without exploiting or disrupting the application.

The assessment identified several configuration-related security issues, primarily involving missing security headers, insecure cookie attributes, and transport security weaknesses. These findings are common in many web applications and do not indicate an active compromise. However, if left unaddressed, they may increase exposure to client-side attacks, data leakage, or user trust issues over time.

Overall, the website demonstrates a functional but improvable security posture. Implementing basic security best practices would significantly strengthen defenses and reduce the overall attack surface.